

Git & GitHub Cheatsheet — Daily Use

Setup (one-time)

```
bash

git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Starting a project

```
bash

git init                # start a new repo here
git clone <url>         # copy a remote repo locally
```

The basic daily loop

```
bash

git status              # what's changed?
git add <file>          # stage a specific file
git add .               # stage everything
git commit -m "message" # save a snapshot
git push                # send commits to remote
git pull                # get latest from remote
```

Checking history

```
bash

git log                 # full history
git log --oneline       # compact history
git log --oneline --graph --all # visual branch history
git diff                # unstaged changes
git diff --staged       # staged changes
git show <commit>       # details of one commit
```

Branching

bash

```
git branch                # list local branches
git branch <name>         # create a branch
git checkout <name>       # switch to a branch
git checkout -b <name>    # create + switch in one step
git switch <name>         # switch (newer alternative)
git switch -c <name>      # create + switch (newer alternative)
git branch -d <name>      # delete a merged branch
git branch -D <name>      # force-delete a branch
```

Merging & rebasing

bash

```
git merge <branch>        # merge branch into current
git rebase <branch>       # replay commits onto another base
git rebase -i HEAD~3      # interactive rebase, last 3 commits
```

Undoing things

bash

```
git restore <file>        # discard unstaged changes to a file
git restore --staged <file> # unstage a file (keep changes)
git checkout -- <file>    # old-style discard changes
git reset --soft HEAD~1   # undo last commit, keep changes staged
git reset --mixed HEAD~1  # undo last commit, keep changes unstaged
git reset --hard HEAD~1   # undo last commit, DELETE changes (danger)
git revert <commit>       # make a new commit that undoes one
git commit --amend -m "new msg" # edit the last commit message
git commit --amend --no-edit # add staged changes to last commit
```

Stashing (save work without committing)

bash

```
git stash                # shelve current changes
git stash list           # see all stashes
git stash pop            # reapply + remove latest stash
git stash apply          # reapply but keep in stash list
git stash drop           # delete a stash
```

Remotes

bash

```
git remote -v           # list remotes
git remote add origin <url> # link a remote
git push -u origin <branch> # push + set upstream tracking
git push origin <branch>   # push a specific branch
git fetch                # download changes, don't merge
```

Tags

bash

```
git tag                 # list tags
git tag v1.0.0          # create a tag
git push origin v1.0.0  # push a single tag
git push origin --tags  # push all tags
```

GitHub CLI () — common ones

bash

```
gh auth login           # log in
gh repo create          # create a new repo
gh repo clone <owner>/<repo> # clone a repo
gh pr create            # open a pull request
gh pr list              # list open PRs
gh pr checkout <number> # check out a PR locally
gh pr merge <number>    # merge a PR
gh issue create         # create an issue
gh issue list           # list issues
```

Handy combos & fixes

bash

```
git add -A && git commit -m "msg"      # stage everything + commit
git pull --rebase                       # pull without a merge commit
git log --author="name"                # filter commits by author
git clean -fd                          # remove untracked files/folders
git cherry-pick <commit>                # apply one commit onto current branch
git blame <file>                       # who changed each line, and when
```

Typical daily flow

bash

```
git checkout main
git pull
git checkout -b feature/my-change
# ...make changes...
git add .
git commit -m "Add my change"
git push -u origin feature/my-change
gh pr create
```

Tip: if you only remember one thing — `git status` before and after everything tells you what's really going on.